39. Config Automation Mindset (Ansible-oriented)

39. Config Automation Mindset (Ansible-oriented)

Phase: Phase 11 — Containers and cloud Linux
Estimated time: 1–2 days
Slug: `39-automation-ansible`

Learning objectives

- Prefer idempotent config management
- Separate inventory from playbooks conceptually
- Avoid snowflake servers
- Combine with immutable images when appropriate

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

A server configured by hand cannot be recreated reliably. Two engineers run slightly different commands, and a month later nobody remembers what differs between staging and production. **Configuration management** writes the desired state down as code, applies it to many hosts, and can be run again and again. Ansible is popular because it needs only SSH and Python on the target (agentless) and uses readable YAML. The key concept is **idempotency**.

Core concepts

Term	Meaning
inventory	List of hosts and groups to manage
module	A unit of work (`apt`, `copy`, `service`, `user`)
task	One module call with arguments
play	Tasks applied to a group of hosts
playbook	YAML file with one or more plays
role	Reusable bundle of tasks, templates, defaults
handler	Task that runs only when notified by a change (restart on config change)
fact	Collected host information (`ansible_os_family`)
check mode	Dry run with `--check`, optionally `--diff`

Idempotent means **same result if repeated**: running the playbook once or ten times leaves the host in the same state, and later runs report “ok” rather than “changed”.

```
 playbook run 1:  [changed] [changed] [changed]   (system converges)
 playbook run 2:  [ok]      [ok]      [ok]        (nothing to do)
 someone edits server by hand
 playbook run 3:  [ok]      [changed] [ok]        (drift corrected)
```

Compare commands:

```
 non-idempotent:  echo "line" >> /etc/app.conf      (adds a line each time)
 idempotent:      lineinfile / template modules      (ensures the line exists
once)
 non-idempotent:  useradd bob                        (fails the second time)
 idempotent:      user: name=bob state=present
```

Worked examples

Inventory, `inventory.ini`:

```
[web]
web1 ansible_host=203.0.113.10 ansible_user=deploy
web2 ansible_host=203.0.113.11 ansible_user=deploy

[db]
db1 ansible_host=203.0.113.20 ansible_user=deploy
```

Ad hoc checks:

```
ansible all -i inventory.ini -m ping
ansible web -i inventory.ini -m command -a 'uptime'
ansible web -i inventory.ini -b -m apt -a 'name=nginx state=present
update_cache=yes'
```

A playbook, `site.yml`:

```
- hosts: web
  become: true
  vars:
    app_port: 8080
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
    - name: Render site config
      ansible.builtin.template:
        src: site.conf.j2
        dest: /etc/nginx/conf.d/site.conf
        mode: "0644"
      notify: Reload nginx
    - name: Ensure nginx running and enabled
```

```
    ansible.builtin.service:
      name: nginx
      state: started
      enabled: true
  handlers:
    - name: Reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

Template, `site.conf.j2`:

```
server { listen {{ app_port }}; location / { return 200 "ok from
{{ inventory_hostname }}\n"; } }
```

Run, preview, and prove idempotency:

```
ansible-playbook -i inventory.ini site.yml --check --diff
ansible-playbook -i inventory.ini site.yml
ansible-playbook -i inventory.ini site.yml     # second run: changed=0
```

Expected recap on the second run:

```
web1 : ok=4 changed=0 unreachable=0 failed=0
```

Limit and debug:

```
ansible-playbook site.yml -l web1 -v
ansible-playbook site.yml --tags config --start-at-task "Render site config"
ansible-lint site.yml
```

Avoid shell where a module exists; if you must use `command`, add a guard:

```
- name: Initialize database once
  ansible.builtin.command: /opt/app/init-db.sh
  args:
    creates: /var/lib/app/.initialized
```

Mini-lab

1. Use a local VM or container as the target with SSH and Python.
2. Write the inventory and run the `ping` module.
3. Create a playbook that installs a package, writes a file via `template`, and enables a service.
4. Run twice. Confirm the second run shows `changed=0`.
5. Manually edit the managed file on the target. Run again and observe the drift fix.
6. Replace one task with `shell: echo x >> file`. Run twice and see why it is not idempotent. Fix with `lineinfile`.

Decision guide / failure modes

Need	Choose
Bootstrap new VM	cloud-init first, Ansible afterwards
Config for many hosts	Ansible playbooks/roles in git
Immutable deploys	build images; use Ansible to build them
One-off fix across fleet	ad hoc command, then codify it
Secrets	Ansible Vault or external secret store

Failure modes:

- `shell` and `command` always report changed unless guarded with `creates`, `removes`, or `changed_when`.
- Editing hosts by hand makes drift; the next run may undo your hotfix.
- Forgetting `become: true` for root-only tasks.
- Typos in YAML indentation; run `ansible-playbook --syntax-check`.
- Unpinned package versions make “same playbook, different result”.
- Running against production without `--check` and `-l` limits first.

Common mistakes

- Manual SSH drift with no record
- One mega playbook without checks
- Automating before understanding

Exercises

Work in `exercises/39-automation-ansible/`. Read that folder’s `README.md` fully before coding.

```
cd exercises/39-automation-ansible
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `idempotent_means` in `exercise.sh` so it prints the hyphenated definition used in this lesson for an idempotent operation. Use all lowercase letters with hyphens between words and no spaces. Afterwards, list two shell commands from your own history that are not idempotent and write the module or guard you would use to make each of them safe to repeat. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/39-automation-ansible/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What is idempotency?
2. Why avoid snowflakes?
3. SSH role in Ansible?

Further reading

- https://docs.ansible.com/